RubberDuck

Detection of Subtle Logic Bugs with developer intent

AI-Assisted Software Engineering | Final Presentation

Borrows from rubber duck debugging: explaining code out loud to catch mistakes. RubberDuck automates that and replaces the silent object for an agent that can see the code and ask a real question back.

Nvafeomo

The Problem

Already caught by IDEs & static analyzers

- Syntax errors
- Type mismatches
- Null dereferences

Misses code that's valid but not what's meant

- Using the wrong dictionary key
- Iterating over the wrong list
- Performing an operation on the wrong variable

Motivation & Who Benefits

Why it matters

- Easy to fix but hard to find
- More costly the further downstream it's caught

Who benefits

- Solo developers & small teams - no dedicated reviewers
- Anyone working across many files, where it's easy to miss things
- Less useful for larger teams with QA In their release processes

Course Perspective

AI4SE -- primary

The LLM supports debugging and code review during development, but it doesn't replace the developer's review step.

Sitting at the pre-commit hook, it checks code against what the developer meant to do.

Current Practice & Existing Solutions

Static analyzers & linters	Catch rule-based issues (style, types, unused code) - no intent
DeepBugs (Pradel & Sen, 2018)	Closest prior work - ML on artificial bugs, predates LLMs, no task input
GitHub Copilot debugging	Explains and fixes errors, crashes, and failing tests: usually need a symptom first
AI code review (Copilot, CodeRabbit)	Reviews PRs and local changes, on request, without a intent
RubberDuck	Runs automatically at commit and checks staged code against the stated intent

Where RubberDuck Has the Edge

No symptom needed	Copilot's debugging tools start from an error or failing test. Logic slips run cleanly.
Per-change intent	Asks what this change should do.
Automatic at commit	Runs itself in the pre-commit hook. Copilot and CodeRabbit local reviews run on request.

Where others are stronger

They check for more kinds of problems (security, speed, style), work in many languages, and plug right into your editor and pull requests

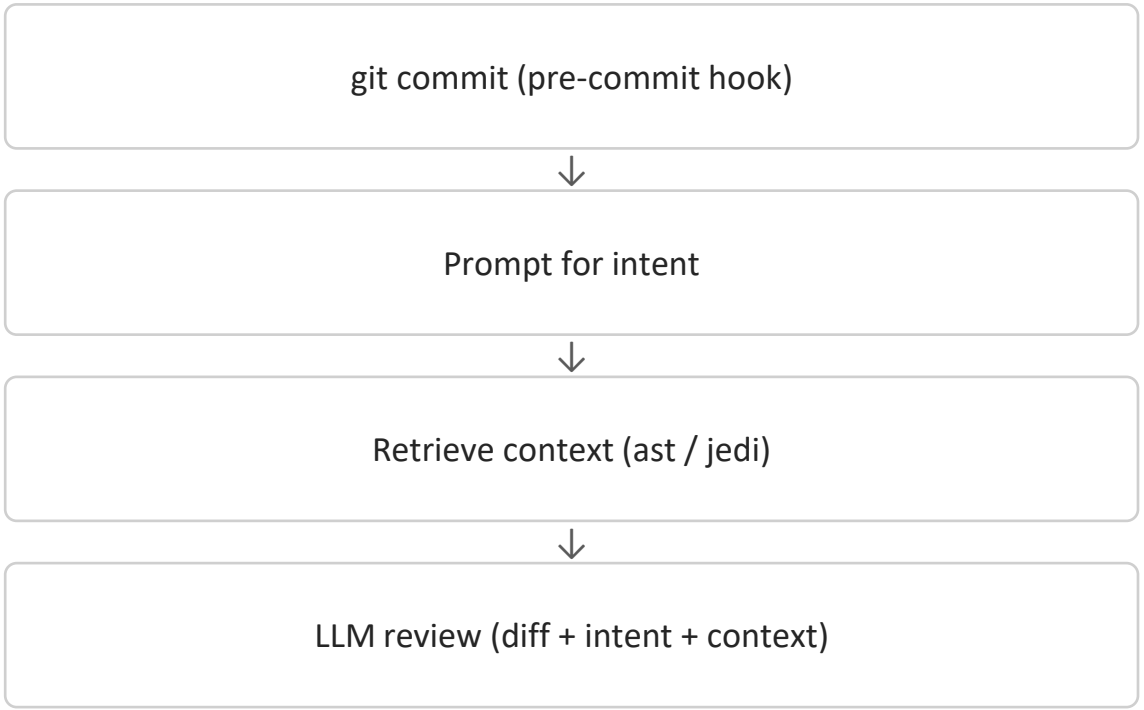
How It Works



Output: a clean pass, or a small number of targeted questions

Never makes assertions - RubberDuck only has partial context, so it asks (“Line 42: did you mean X or Y?”) rather than declaring code wrong.

System Workflow & Tech Stack



Tech stack

Trigger: Git pre-commit hook or a one-word command (Python)

Context retrieval: ast module + jedi

LLM / agent: Gemini 3.5 Flash-Lite, one call

Bug seeding: Custom mutations (wrong variable, wrong key, loop swaps)

Evaluation: built-in eval on TinyDB (recall, precision, false positives)

Implementation Details

Python, Git-aware tool	Runs from a pre-commit hook
Code input	Staged diff marks changed lines, but full file is taken
Cross-file context	Jedi resolves names used in the change and adds their definitions from other files
LLM review call	One Gemini 3.5 Flash-Lite request with the diff, file, intent, and related code
Output	Up to three questions or a clean pass. Can abort the commit or bypass and commit anyway.

Evaluation

Does developer intent + cross-file context improve bug-detection accuracy over the same agent without it?

Setup

15 bugs seeded into TinyDB (5 per bug type) plus the same functions left clean. Each docstring stands in for the intent.

Conditions	Full vs context-only vs intent-only vs code-only (baseline: same call, no intent or context)
Accuracy	Recall (bugs caught), precision (flags that are right), false-positive rate on clean code

Results

Condition	Recall	Precision	False alarms (clean)	Avg latency
Full (intent + context)	**14/15 (0.93)**	**0.88**	**2/15 (0.13)**	**0.92 s**
Context only	14/15 (0.93)	0.93	1/15 (0.07)	0.93 s
Intent only	14/15 (0.93)	0.93	1/15 (0.07)	0.99 s
Code only (baseline)	14/15 (0.93)	0.88	2/15 (0.13)	0.88 s

14 of 15 bugs caught in every condition: on these bugs, intent and context made no measurable difference

By bug type (same in all conditions): wrong variable 5/5, wrong loop collection 5/5, wrong dictionary key 4/5. The one miss was the same dictionary-key swap every time, and the precision gaps come down to a single false alarm.

What Was Delivered

Built and working

- Pre-commit hook that asks for intent and reviews code
- Jedi-based cross-file context retrieval
- Automatic retries and fallback models when the API is busy
- Asks questions about potential issues in terminals

Limitations & lessons learned

- Python only
- No UI
- Questions are only as good as the stated intent
- Free-tier API overloads led to retries and backup models
- 15 easy-to-spot bugs were too few to show a benefit from intent

Takeaway

Not a replacement for the developer

RubberDuck checks staged code against what the developer meant, and asks before the commit

---

Why it helps	Quick conversation before the commit to avoid a costly debug session later
What we found	Caught 14 of 15 seeded bugs in about a second per review; intent and context still need a harder test

References & External Resources

Paper	M. Pradel & K. Sen, “DeepBugs: A Learning Approach to Name-based Bug Detection,” OOPSLA 2018
API	Google Gemini API Documentation • ai.google.dev/gemini-api/docs
Corpus	TinyDB (evaluation corpus for seeded bugs) • github.com/msiemens/tinydb

Live Demo



The agent's own stated intent becomes RubberDuck's check on the agent's code

Planted slips: a wrong dictionary key, a loop over the wrong list, and a wrong variable. All valid code that still runs.

Thank You

Questions?